Laboratorio II
¿Qué va a ser un higuerón?

IC-2001 Estructuras de datos
Valor: 7 %
Prof. Eddy Ramírez

Intrucciones generales

1. El laboratorio es individual.

   Cualquier sospecha de fraude se asignará nota mínima en el curso y se aplicará el reglamento del TEC para estos casos.

2. Debe desarrollar los ejercicios indicados utilizando C++ sobre GNU/Linux, haciendo una clase con sus respectivos métodos.

3. Ninguna función puede constar de más de 25 líneas.

   Si requiere implementar funciones auxiliares para poder cumplir de forma adecuada a lo que se le solicita puede (y debe) hacerlo. Escriba nombres significativos para las funciones (recomendado) y utilice comentarios para cada método.

4. Asegúrese de cumplir con lo solicitado en el enunciado de cada ejercicio, que el dominio y el codominio sean correctos para cada método.

5. En el desarrollo de este laboratorio sólo puede utilizar bibliotecas de C++: math, iostream y string (pero no set)y el namespace std.

6. Todas las soluciones deben estar implementadas de forma **eficiente** (Bajo el marco de lo comprendido en el curso hasta el momento).

7. Si el programa no compila, se asignará una nota de cero. La versión que se utilizará será C++14 o superior.

1. Objetivos del laboratorio

1.1. Objetivo general

Brindar al estudiante la capacidad de abstracción y la comprensión de estructuras de datos y su uso para solucionar problemas. Brindar visión y las herramientas fundamentales para poder ingresar a tecnologías y temas computacionales más avanzados donde estos conocimientos son pilares.

1.2. Objetivos específicos

1. Implementar soluciones utilizando las diversas estructuras de datos aprendidas

2. Explicar distintas estructuras de datos (jerárquicas) con el objetivo de la resolución de problemas de mediana complejidad en la implementación y diseño.

3. Elaborar estructuras de datos alternas que permitan la resolución de problemas con mayor eficacia, haciendo híbridos con el objetivo de crear soluciones más robustas.

2. Especificación

Este proyecto consiste en elaborar la clase SET que significa conjunto en inglés, va a estar compuesto únicamente por elementos de tipo string, el orden, es irrelevante, pero no debe haber elementos repetidos. Deben de poder implementarse los métodos descritos en la sección 2.1, pero utilizando como único apoyo AVL y doble hash.

2.1. Métodos de la clase SET

1. Constructor SET (int cap)

   Si no se envía parámetro, se hace un set con una capacidad indefinida, caso contrario se hace un SET cuya capacidad máxima es *cap*.

2. int estaVacio()

   Es un valor booleano, indica si el conjunto está vacío

3. int cardinalidad()

   Retorna la cantidad de elementos que tiene el SET

4. int cardinalidadMax()

   Retorna la cantidad máxima de elementos que le caben al SET, $2^{31} - 1$, si no estuviera definida.

5. bool insertar(string s)

   Dominio: Un string s

   Codominio: Retorna true si fue exitosamente insertado y false si no (porque ya estaba o porque estaba lleno)

   Comportamiento, el string s debe estar insertado en SET a menos que esté lleno.

6. bool eliminar(string s)

   Dominio: Un string s

   Codominio: Retorna true si fue exitosamente eliminado y false si no (porque no estaba)

7. bool contiene(string s)

   Dominio: Un string s

   Codominio: true si s está en el SET o false si no.

8. SET intersect(SET conj) //intersección

9. SET union(SET conj) //union

10. Lista elementos()

    Retorna una lista con el *preorden* del árbol AVL de apoyo

3. Detalles de implementación

3.1. Nodo AVL

El nodo del AVL estará compuesto (además de los dos punteros), por un string y por dos enteros, h1 y h2 que serán los que se utilicen como índices de hash. El h1 debe ser el entero que sirva además como llave para indicar si un string es "mayor.º "menor"que otro.

3.2. Árbol AVL

El árbol AVL[1], es un árbol binario que siempre busca estar balanceado. La idea es tomar este árbol como base para implementar el SET.

Si se inserta un elemento nuevo, se inserta exactamente como lo dicta el árbol, tal cual se hace con un árbol binario de búsqueda (BST en inglés) y luego balanceando con giros simples o dobles según corresponda. Para eliminar se haría igual, procurando que el factor de balanceo de los nodos estén correctos. Para saber si un elemento está en al AVL, bastaría con recorrerlo e indicar si el elemento se encuentra. Cuando se inserta un elemento que ya había sido insertado previamente.

En el caso de la función unión, lo que debe hacerse es al SET con mayor tamaño se le inserta cada uno de los elementos del SET con menor tamaño.

Finalmente para el concat, para cada elemento del SET con menor tamaño, se busca si está en el otro SET, de estarlo se ingresa en el SET respuesta.

3.3. Doble Hash

Para poder comparar si dos strings son iguales (o una “menor” que la otra), en lugar de tener que hacer una comparación char a char, se le va asignar a cada nodo dos números de hash h1 y h2. Es importante que esta función cumpla con las condiciones de ser uniformemente distribuidas. Entonces antes de comparar char a char, se valida que los dos valores h1 y h2 (asignados por las funciones respectivas) sean diferentes. (En

[1]Deben su nombre a Georgii Adelson-Velskii y Yevgeniy Landis

principio, sólo si el elemento es potencialmente el mismo, h1 y h2 van a ser iguales). El valor de h1 se va a utilizar para saber si un string debe ir a la derecha o a la izquierda de un nodo en el AVL.

3.3.1. Función de hash

Se les recuerda que esta es una forma genérica de una función válida de hash:

```
int hash(string s, int p, int q){
     int h = 0;
     for(int i = 0; s[i]; i++, h%=p)
          h+=expt(q,s[i],p);
     return h%p;
}
```

Donde los números p y q son primos (tanto mejor si $q < p$) y expt es la función (desde luego, con exponenciación logarítmica) tal que: $expt(a, b, c) \equiv a^b(c)$

4. Palabras inusuales

Resuelva el siguiente problema utilizando la estructura de datos creada en la sección anterior:

Todos las publicaciones de Juanito en la red social Y están conformadas por una cadena de palabras en minúscula separadas por espacios. Al revisar dos de sus publicaciones, Juanito quiere saber cuáles son las palabras inusuales en estas.

Una palabra inusual es aquella que se encuentra en una publicación y no en la otra, pero que además no se repite en una misma publicación

Dado un entero C que representa la cantidad de casos de prueba, dos enteros N y M que se refieren a la cantidad de palabras en cada publicación y dos strings que son las publicaciones como tal, imprima las palabras inusuales en las publicaciones de Juanito en orden alfabético. En caso de no haber palabras inusuales imprima -1.

4.1. Ejemplo de entrada

```
2
5 5
no me gusta el queso
tampoco me gusta el oregano
5 5
el carro es el problema
otro problema de mi carro
```

4.2. Ejemplo de salida

```
no
oregano
queso
tampoco
de
es
mi
otro
```

5. Evaluación

Este laboratorio no requiere interfaz gráfica. La evaluación va a estar dada por la eficacia y eficiencia de los métodos especificados. Es importante que si algún rubro en la evaluación falla, se afectarán los siguientes rubros de la evaluación.

Para probar el programa se debe leer de la consola una serie de entradas, utilizando la clase SET programada por usted, se debe crear un main que haga lo siguiente:

Debe leer una primer línea con el número de instrucciones que se deben a leer a continuación (una línea por instrucción)

Cada instrucción puede ser de la forma:

- E CANT

  Donde E indica que se inicia el conjunto con tamaño CANT.

- i s

  Donde i significa insertar y s es el string que se desea insertar, muestra "SI insertado" si se pudo insertar o "NO insertado" si no se pudo.

- r s

  Donde i significa remover y s es el string que se desea remover, muestra "SI removido" si se pudo remover o "NO removido" si no se pudo.

- c

  Debe mostrar la cantidad de elementos en el conjunto

- v

  Debe mostrar "VACIO" o "NO VACIO" si el conjunto está vacío o no.

- t s

  Donde t indica si el conjunto "tiene" al string s. Imprime "SI ESTA" si el elemento está o "NO ESTA" si el elemento no está.

5.1. Ejemplo de entrada

```
E 3
i primero
i segundo
i tercero
i cuarto
r cuarto
r segundo
r primero
c
v
t tercero
t segundo
```

5.2. Ejemplo de salida

```
SI insertado
SI insertado
SI insertado
NO insertado
NO removido
SI removido
SI removido
1
NO VACIO
SI ESTA
NO ESTA
```

Por otra parte, se debe resolver el problema *Palabras inusuales* planteado en la sección 4. Para esto, debe crear un main aparte del anterior.

El porcentaje de líneas contestadas correctamente contra casos de prueba, se multiplicará por la rúbrica aquí descrita.

Producto	**Valor**
Nodo AVL con hash	2.5 %
Algoritmo árbol AVL	10 %
Funciones de SET	25 %
Palabras inusuales	50 %
Documentación	12.5 %

La documentación de este laboratorio consiste en responder estas preguntas:

1. ¿Por qué se utilizó un AVL para implementar un SET en lugar de una lista?

2. ¿Qué ventajas tiene la implementación de un SET con una lista y qué ventajas tiene la implementación de un SET con un AVL?

3. ¿Fue la función de hash útil para clasificar las string? Justifique

4. ¿Qué otras soluciones (buscando ante todo la eficiencia) son posibles para implementar un SET?

Las preguntas deben ser contestadas en LaTeX y entregadas en un pdf con los apellidos y el nombre del autor. Ejemplo: *ramirezjimenezeddy.pdf*

6. Formato de Entrega

1. El laboratorio debe entregarse a los correos *anamora*02@*estudiantec.cr* y *edramirez@tec.ac.cr* antes de las 12:00 mn del miércoles 4 de setiembre. El asunto: **IC-2001 Estructuras de Datos - Laboratorio II[Nombre Estudiante]**.

2. Los archivos deben estar comprimidos en formato zip o tar.gz y tener un MAKEFILE.

3. El nombre del archivo ZIP debe ser los dos apellidos, seguido del nombre y el número de laboratorio. Ejemplo: **RamírezJiménezEddy2.zip**